

type from the variable's first assignment statement. For example, from the line `spam = 42`, the type checker can infer that `spam` is supposed to have a type hint of `int`. But I recommend setting a type hint anyway. A future change to a `float`, as in `spam = 42.0`, would also change the inferred type, which might not be your intention. It's better to force the programmer to change the type hint when changing the value to confirm that they've made an intentional rather than incidental change.

Using Static Analyzers

Although Python supports syntax for type hints, the Python interpreter completely ignores them. If you run a Python program that passes an invalidly typed variable to a function, Python will behave as though the type hints don't exist. In other words, type hints don't cause the Python interpreter to do any runtime type checking. They exist only for the benefit of static type-checking tools, which analyze the code before the program runs, not while the program is running.

We call these tools *static analysis* tools because they analyze the source code before the program runs, whereas runtime analysis or dynamic analysis tools analyze running programs. (Confusingly, *static* and *dynamic* in this case refer to whether the program is running, but *static typing* and *dynamic typing* refer to how we declare the data types of variables and functions. Python is a dynamically typed language that has static analysis tools, such as Mypy, written for it.)

Installing and Running Mypy

Although Python doesn't have an official type-checker tool, Mypy is currently the most popular third-party type checker. You can install Mypy with `pip` by running this command:

```
python -m pip install --user mypy
```

Run `python3` instead of `python` on macOS and Linux. Other well-known type checkers include Microsoft's Pyright, Facebook's Pyre, and Google's Pytype.